JavaScript

Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Davide Loconte

Gabriele Colapinto

April 17, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.
https://creativecommons.org/licenses/by-nc/4.0/

© 2025 Gabriele Colapinto
Released under the Creative Commons Attribution–NonCommercial 4.0 International License

Contents

1 Core Concepts and Programming Paradigm **5**
1.1 JavaScript Engine: AST, Bytecode, JIT, and De-optimization . . . 6
1.1.1 Abstract Syntax Tree (AST) . . . 6
1.1.2 Bytecode Generation . . . 6
1.1.3 Interpretation . . . 7
1.1.4 Just-In-Time Compilation (JIT) . . . 7
1.1.5 Speculative Optimization . . . 7
1.1.6 De-optimization . . . 7
1.1.7 Coexistence of Interpreted and Compiled Code . . . 8
1.1.8 Concurrency and External Interactions . . . 8
1.1.9 Platform Dependence . . . 8

2 Syntax and Integration with HTML **9**

3 Variables and Data Types **11**
3.1 Variable Declaration: `var`, `let`, and `const` . . . 11
3.2 Primitive Data Types . . . 12
3.2.1 Strings and Implicit Type Coercion in JavaScript . . . 12

4 Operators and Control Flow **15**
4.1 Operators . . . 15
4.1.1 Loose equality vs. strict equality . . . 15
4.1.2 Complex logical operators in JavaScript . . . 15
4.1.3 Clarification of Bitwise Operations . . . 17
4.1.4 Clarification of the Ternary Operator . . . 24
4.1.5 Clarification of Type Operators . . . 25
4.2 Control Flow Statements . . . 29
4.2.1 Conditional Statements . . . 29
4.2.2 Loop Statements . . . 29
4.2.3 Truthy and Falsy Values . . . 30

5 Functions: Reusable Blocks of Code **33**
5.1 Function Definition and Usage . . . 33

6 Working with Data Structures: Arrays and Objects **35**
6.1 Arrays . . . 35
6.1.1 Example of `forEach()` usage . . . 37
6.1.2 Example of `map()` usage . . . 38
6.2 Objects . . . 38
6.2.1 Creating an object . . . 39
6.2.2 Methods . . . 40
6.2.3 Common objects . . . 40

7 Advanced Concepts: this and Symbol **43**
7.1 The `this` Keyword: Understanding Context 43
7.1.1 Strict mode vs. non-strict mode 43
7.1.2 The "left of the dot" rule 43
7.1.3 Default binding (global context) 44
7.1.4 The `this` problem inside callbacks 44
7.1.5 Solutions to preserve `this` 45
7.1.6 Key observations 46
7.1.7 Important takeaway 46
7.2 The Symbol Type: Ensuring Uniqueness 46

1 Core Concepts and Programming Paradigm

JavaScript is a dynamically typed, platform-independent scripting language designed for client-side execution within a web browser. Introduced in 1995 with Netscape Navigator 2.0, its strategic importance lies in its ability to transform static web pages into active, interactive user experiences. As a core technology of the web, JavaScript works alongside HTML and CSS to provide the dynamic behavior that defines modern applications. Despite the name, JavaScript is not related to the Java programming language. Unlike many traditional languages, it is built on a prototype-based object-oriented paradigm, a foundational concept that shapes how data and behavior are structured.

A key distinction in object-oriented languages is the difference between class-based and prototype-based models. Mainstream languages like Java and C++ use a class-based system, whereas JavaScript employs a prototype-based approach. It's important to note that while modern JavaScript includes a `class` keyword for a more familiar syntax, this is merely "syntactic sugar" over the underlying prototype system and does not change its fundamental paradigm. The core differences are summarized below:

Paradigm	Entity Model	Property Inheritance
Class-based (e.g., Java, C++)	Based on two distinct entities: **classes** (blueprints) and **instances** (objects created from the blueprint). A class defines the properties and methods for a set of objects.	An instance inherits properties directly from the class it was created from. The class acts as the single template.
Prototype-based (JavaScript)	Based on a single entity: the **object**. There is no formal distinction between a class and an instance.	A new object inherits properties from another object that serves as its **prototype**. Any object can act as a template for another.

Modern web browsers execute JavaScript code using sophisticated engines, such as Google's V8 (used in Chrome) and Mozilla's SpiderMonkey (used in Firefox). These engines do not simply interpret code line-by-line. Instead, they employ a process known as **Just-In-Time (JIT) compilation**. This technique compiles JavaScript into executable machine code at runtime, making execution significantly more efficient and faster than traditional interpretation.

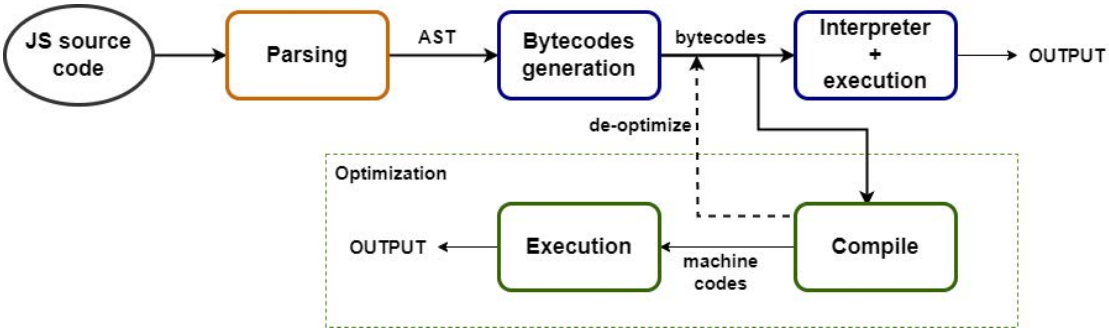

Figure 1.1: JavaScript engine compilation pipeline

1.1 JavaScript Engine: AST, Bytecode, JIT, and De-optimization

Figure 1.1 illustrates the execution pipeline of a modern JavaScript engine (e.g., V8), which combines interpretation and Just-In-Time (JIT) compilation to achieve both fast startup and high performance.

1.1.1 Abstract Syntax Tree (AST)

After parsing, the JavaScript source code is transformed into an **Abstract Syntax Tree (AST)**. It is called "abstract" because it represents the logical structure of the program while omitting irrelevant syntactic details such as whitespace, formatting, or redundant parentheses.

For example:

```
1 + 2 * 3
```

is represented as a tree where nodes correspond to operations and operands, reflecting operator precedence.

```
  +
 / \
1   *
   / \
  2   3
```

The AST captures the semantics of the program rather than its textual form.

1.1.2 Bytecode Generation

From the AST, the engine produces **bytecode**, an intermediate representation that is closer to execution than source code but still independent of the underlying hardware.

The bytecode:

- is **engine-specific** (e.g., V8 bytecode),
- is **independent of the operating system and CPU**,
- enables portability across platforms running the same engine.

1.1.3 Interpretation

Initially, the bytecode is executed by an **interpreter**. This allows the program to start quickly without the overhead of compiling everything upfront. The interpreter executes instructions one by one, which is flexible but slower than native execution.

1.1.4 Just-In-Time Compilation (JIT)

To improve performance, modern engines monitor execution and identify **hot code**, i.e., portions of the program that are executed frequently (such as loops or frequently called functions).

These parts are compiled into **machine code** by a JIT compiler. The compiled code runs significantly faster than interpreted bytecode.

The selection of code to compile is based on heuristics such as:

- execution frequency,
- stability of observed types,
- predictability of control flow.

1.1.5 Speculative Optimization

JIT compilation relies on **speculative assumptions**. For instance, if a function is repeatedly called with numeric arguments, the compiler may generate optimized machine code assuming numeric types only.

These optimizations include:

- type specialization,
- function inlining (inlining = embedding directly within HTML),
- inline caching,
- optimized object access.

1.1.6 De-optimization

If a speculative assumption is violated (e.g., a function previously called with numbers is later called with strings), the optimized machine code may no longer be valid.

In this case, the engine performs **de-optimization**:

- execution falls back to the interpreter,
- a more general (less optimized) version of the code is used.

This explains the feedback loop shown in the figure, where compiled code can return to interpreted execution.

1.1.7 Coexistence of Interpreted and Compiled Code

Interpreted and compiled code can coexist seamlessly. Different parts of the same program may be:

- interpreted,
- compiled and optimized,
- de-optimized and reinterpreted.

This does not introduce inconsistencies because the engine preserves the same JavaScript semantics across all execution modes.

1.1.8 Concurrency and External Interactions

The presence of JIT compilation does not affect the concurrency model of JavaScript. The language remains:

- single-threaded (based on the event loop),
- asynchronous for I/O operations.

Therefore, concerns such as race conditions or external interactions (e.g., network requests) are not impacted by whether code is interpreted or compiled.

1.1.9 Platform Dependence

While bytecode and optimization strategies are largely platform-independent, the final **machine code** generated by the JIT compiler is platform-specific.

Thus:

- the overall execution strategy is consistent across platforms,
- low-level optimizations may vary depending on the CPU architecture.

With this conceptual background established, we can now move to the practical steps of embedding and writing JavaScript code within a standard web page.

2 Syntax and Integration with HTML

To bring a web page to life with JavaScript, the code must be embedded within an HTML document. The browser's JavaScript engine will then discover and execute this code as it parses the page. This section provides the practical foundation for writing and executing your first lines of JavaScript.

There are two primary methods for including JavaScript in an HTML file:

1. **Internal Script:** You can write JavaScript code directly within an HTML document by placing it inside a `<script>` tag. This is suitable for small, page-specific scripts.

```
<script type="text/javascript">
    ... JavaScript code ...
</script>
```

2. **External Script:** For better organization and code reuse, JavaScript can be placed in a separate file with a `.js` extension. This file is then linked to the HTML document using the `src` attribute of the `<script>` tag. This approach is highly recommended as it separates structure (HTML) from behavior (JavaScript), improving maintainability.

```
<script type="text/javascript" src="myfunctions.js"/>
```

To provide a fallback for users who have disabled JavaScript in their browsers, or for non-interactive user agents like **printers** that cannot execute code, the `<noscript>` tag is used. The content inside `<noscript>` is displayed only when scripts are not supported or are disabled.

```
<noscript>
  <p>
    This website requires JavaScript to be enabled for full functionality.
  </p>
</noscript>
```

JavaScript's syntax is heavily influenced by C-like languages and each segment of JS code is formed by one or more statements. Key syntactical rules include:

- Statements can be separated by a semicolon (`;`), although a line break is often sufficient to mark the end of a statement.
- The language is **case-sensitive**, meaning `myVariable` and `myvariable` are treated as two different variables.
- Comments can be written in two ways:
  - Single-line comments start with `//`.

```
// This is a single line comment
```

- Multi-line comments are enclosed between `/*` and `*/`.

```
/*
    This is
    a multi-line
    comment
*/
```

Understanding this basic syntax allows us to move on to the fundamental building blocks of any program: variables.

3 Variables and Data Types

In JavaScript, variables are named storage locations that hold values, which can be changed during a program's execution. A variable name must start with a letter or an underscore. A critical feature of the language is its use of **dynamic typing**, which means a variable's data type is determined at runtime and can change as different values are assigned to it. Understanding how to declare variables and work with their underlying data types is essential for managing information within any application.

3.1 Variable Declaration: `var`, `let`, and `const`

JavaScript provides three keywords for declaring variables: `var`, `let`, and `const`. While they may appear similar at first, they differ significantly in their scoping, re-declaration, and re-assignment rules.

Keyword	Scope	Re-declaration & Re-assignment Rules
`var`	**Global or Function Scope:** Variables are accessible throughout the function in which they are declared, or globally if declared outside any function.	Can be **re-declared** and **re-assigned** (updated) within the same scope.
`let`	**Block Scope:** Variables are only accessible within the block (`{...}`) in which they are declared (e.g., inside an `if` statement or `for` loop).	Can be **re-assigned** but **cannot be re-declared** within the same scope.
`const`	**Block Scope:** Variables are only accessible within the block (`{...}`) in which they are declared.	**Cannot be re-assigned or re-declared**. Must be initialized at the time of declaration.

Another crucial difference lies in a behavior called **hoisting**, where variable declarations are conceptually moved to the top of their scope during the compilation phase.

- Variables declared with `var` are hoisted and automatically initialized with the value `undefined`. This allows them to be accessed before their actual declaration line without causing an error (though their value will be `undefined`).
- Variables declared with `let` and `const` are also hoisted, but they are not initialized. They exist in a state known as the "Temporal Dead Zone" (TDZ) from the start of their scope until the point of declaration. Accessing them within the TDZ will result in a `ReferenceError`.

For modern JavaScript development, the best practice is to use `const` for any variable whose value should not change after initialization and `let` for variables that will be reassigned. The

`var` keyword is generally avoided to prevent common bugs associated with its less restrictive function-scoping and hoisting rules.

3.2 Primitive Data Types

JavaScript has seven primitive data types, which are the most basic building blocks of data in the language.

- **Number**: Represents both integer and floating-point numbers. Example: `42`, `3.14`.
- **String**: A sequence of characters enclosed in quotes. Example: `"Hello, world!"`.
- **Boolean**: Represents a logical entity with two values: `true` or `false`.
- **Undefined**: Represents a variable that has been declared but has not yet been assigned a value. Examples: `let name;` or `let name = undefined;`. To the ends of the JS interpreter both notations are equivalent but the second notation makes the declaration clearer to the eyes of a human reader.
- **Null**: Represents the intentional absence of any object value. Example: `let data = null;`.
- **Symbol**: A unique and immutable value, introduced in ES6, often used as a key for an object property. Example: `Symbol('id')`.
- **BigInt**: Represents integers of arbitrary precision, used for numbers larger than the `Number` type can handle (Introduced in ES2020). Example: `9007199254740991n`.

A cornerstone concept in JavaScript is that while these seven types are primitives, **all other elements in the language are objects**. This includes data structures like arrays and even functions, which we will explore later.

A common point of confusion is the semantic difference between `Undefined` and `Null`. `Undefined` is the default state of a variable that has been declared but has never seen an assignment; it signals an unintentional absence of a value. In contrast, `Null` is a value that is explicitly assigned to a variable to represent an intentional absence of data.

The `Symbol` data type is a more recent addition, primarily used to create unique identifiers for object properties, which helps prevent accidental name collisions between different parts of a code base.

These data types are the foundation upon which programs operate, and they are manipulated using various operators.

3.2.1 Strings and Implicit Type Coercion in JavaScript

A **string** in JavaScript is a sequence of characters enclosed in quotes. The language provides three ways to define strings:

- **Single quotes** (`'...'`) and **double quotes** (`"..."`): used for simple strings, with no functional difference.
- **Backticks** (`` `...` ``): used for **template literals**, which support:

  - variable interpolation using `${...}`,
  - multi-line strings.

- **Escape characters**: special sequences introduced with a backslash, such as `\n` (newline) or `\"` (escaped quote).

Strings can be combined using:

- **concatenation** with the `+` operator,
- **template literals** for more complex formatting.

```
let s1 = 'Hello';
let s2 = "World";
let s3 = `Hello, ${s2}`;
let s4 = "He said, \"Hello!\"";
let s5 = "Hello, " + "World!";
```

Implicit Type Coercion in String Operations

JavaScript automatically converts values to strings when needed. This process is called **implicit type coercion**.

Consider the following examples:

```
let test  = "Hello " + 10;      // "Hello 10"
let test2 = "Hello " + true;    // "Hello true"
let test3 = `Hello, ${10}`;     // "Hello, 10"
let test4 = `Hello,
${10n}`;                        // "Hello,\n10"
```

In all these cases, non-string values are automatically converted into strings.

How does the conversion work? When the `+` operator is used and at least one operand is a string, JavaScript converts the other operand using an internal operation often referred to as `ToString`.

For example:

- `10` becomes `"10"`
- `true` becomes `"true"`
- `10n` becomes `"10"`

Is autoboxing involved? No. Although JavaScript can temporarily wrap primitives into objects (autoboxing) when accessing methods, implicit string conversion does not require creating wrapper objects such as `new Boolean(true)`.

Instead, conversion is handled internally by the JavaScript engine. Therefore, the following interpretation is conceptually **incorrect**:

```
"Hello " + new Boolean(true).toString()
```

Even though the result is similar, no explicit wrapper object is created.

Template Literals and Backticks

Template literals use backticks (`) and allow embedding expressions:

```
let value = 10;
let str = `Hello, ${value}`;
```

Typing the backtick: Depending on the keyboard layout, the backtick can be typed using a dedicated key (usually near the ESC key) or with `ALT+96` (numeric keypad) on Windows.

Multi-line Strings and Newline Characters

Template literals preserve line breaks:

```
let str = `Hello,
World`;
```

This produces a string containing a newline character:

```
"Hello,\nWorld"
```

Similarly, pressing `SHIFT+ENTER` in the browser console inserts a newline into the string.

How to avoid inserting newline characters If the goal is to split a string across multiple lines in the code without adding newline characters, we must use **string concatenation**:

```
let str = "Hello, " +
        "World";
```

Practical Considerations

Implicit type coercion is a powerful feature but may lead to unintended behavior if not fully understood. For this reason:

- template literals are preferred for readability,
- implicit conversions should be used with care,
- explicit conversions (e.g., `String(value)`) can improve clarity in critical code.

4 Operators and Control Flow

Operators and control flow statements are the core tools for building logic in a JavaScript program. Operators perform actions on data, while control flow statements direct the order of execution based on specific conditions or loops.

4.1 Operators

JavaScript includes a comprehensive set of operators, many of which will be familiar to developers of other C-like languages. They can be grouped into the primary categories contained in table 4.1.

4.1.1 Loose equality vs. strict equality

One of the most critical distinctions to understand is between **loose equality (==)** and **strict equality (===)**. A helpful way to remember the difference is to translate them into questions:

- The **loose equality (==)** operator asks, "Are these values **similar**?" It performs type coercion, meaning it attempts to convert the operands to a common type before making the comparison. For example, `1 == '1'` evaluates to `true` because the string is coerced into a number.
- The **strict equality (===)** operator asks, "Are these values **equal** in both value and type?" It does not perform type coercion. Therefore, `1 === '1'` evaluates to `false` because the operands are of different types (Number and String).

As a best practice, always use the strict equality (`===`) operator to avoid unexpected bugs that can arise from JavaScript's automatic type coercion rules.

4.1.2 Complex logical operators in JavaScript

JavaScript only has the 3 logical operators of the C language which are `&&` (AND), `||` (OR) and `!` (NOT). It does not have more complex logical operators such as XOR, NAND and NOR but they can be derived from the basic ones.

Performing the NAND and NOR operations is trivial because we have to combine the NOT operator and respectively the AND and OR operators. Performing the XOR operations is more complex because we have to combine the 3 main logical operators. Recall that:

$$a\ XOR\ b = (a\ AND\ NOT(b))\ OR\ (NOT(a)\ AND\ b)$$

Operator Type	**Operator**	**Description**	**Example**
Arithmetic	`+`, `-`, `*`, `/`, `%`, `**`	Basic math operations and exponentiation	$5 + 3 \rightarrow 8$, $5 ** 2 \rightarrow 25$
Assignment	`=`, `+=`, `-=`, `*=`, `/=`, `%=`	Assign or update values of the variables	$x = 5$, $x += 2 \rightarrow x = 7$
Comparison	`==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`	Values comparison (Pay attention to the difference between the loose equality and the strict equality)	$5 === 5 \rightarrow true$, $5 != 4 \rightarrow true$
Logical	`&&`, `||`, `!`	Combine or negate boolean expressions	$true \&\& false \rightarrow false$
Bitwise	`&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`	Operate at the bit level	$5 \& 1 \rightarrow 1$, $5 << 1 \rightarrow 10$
Ternary	`condition ? [value if the condition is true] : [value if the condition is false]`	Shorthand for if-else	$5 > 3 \; ? \; 'Yes' : 'No' \rightarrow 'Yes'$
Type	`typeof`, `instanceof`	Check or verify data types	`typeof 'hello'` $\rightarrow$ `string`

Table 4.1: JavaScript operators

a	b	$(a \; AND \; NOT(b))$	$(NOT(a) \; AND \; b)$	$(a \; AND \; NOT(b)) \; OR \; (NOT(a) \; AND \; b)$	$a \; XOR \; b$
`false`	`false`	`false`	`false`	`false`	`false`
`false`	`true`	`false`	`true`	`true`	`true`
`true`	`false`	`true`	`false`	`true`	`true`
`true`	`true`	`false`	`false`	`false`	`false`

Table 4.2: Truth table showing the equivalence of the expression and the XOR operator

??: The Nullish Coalescing Operator

JavaScript has another operator which can be considered a logical OR operator but only for the nullish values (`null` or `undefined`). The `??` operator returns the first operand if it is not

nullish; otherwise, it returns the second operand. For example:

```
let name = null;
let text = "missing";
let result = name ?? text;
```

The third line of the script checks if the first assigned value (the value of `name`) is null and, if so, assigns to `result` the value of the variable `text`. In this case the value of `result` is `"missing"` because the value of the variable `name` is `null`.

The usage of the nullish coalescing operator is a best practice when dealing with variables initialized using the `var` keyword. In fact, they are initialized with an `undefined` value and the nullish coalescing operator provides us a strategy to assign a variable a default value in case the variable under discussion is nullish.

4.1.3 Clarification of Bitwise Operations

Bitwise operators in JavaScript operate directly on the binary representation of numbers. Before applying these operations, JavaScript converts numbers to 32-bit signed integers. This means that bitwise operations are low-level and work at the level of individual bits.

When are bitwise operations useful?

Bitwise operations are particularly useful when:

- representing multiple boolean states compactly,
- managing flags and permissions,
- performing low-level optimizations.

However, in typical JavaScript applications, they are used less frequently because they reduce code readability and JavaScript engines already perform many optimizations automatically.

Performance considerations

Bitwise operations are often considered fast because they map directly to CPU instructions executed by the ALU (Arithmetic Logic Unit). Modern CPUs can perform bitwise operations on entire machine words (e.g., 32 or 64 bits) in a single clock cycle.

However:

- modern JavaScript engines (via JIT compilation) already optimize high-level code efficiently,
- using bitwise operations does not always lead to noticeable performance improvements,
- readability and maintainability are usually more important than micro-optimizations.

JIT optimizations

Modern JavaScript engines (such as V8) can optimize both arithmetic and bitwise operations. In some cases, bitwise operators are used internally by the engine to force values into 32-bit integers, which may enable certain optimizations.

However, the developer should not rely on bitwise operations for performance unless profiling demonstrates a real benefit.

Bitwise operations and memory usage

Bitwise operations can help reduce memory usage when representing multiple boolean values as a single integer. For example, instead of storing multiple boolean variables, a single byte can encode up to 8 independent flags.

Example: Permissions using bit masks

Bitwise operations can be used to handle states and permissions efficiently. A byte can be interpreted as a set of 8 independent flags, where each bit represents a specific permission.

Unix-like operating systems use a similar strategy to represent file permissions.

	r	w	x	Description
User 1	0	0	0	No permissions
User 2	1	0	0	Read only
User 3	0	1	0	Write only
User 4	0	0	1	Execute only
Admin	1	1	1	Full permissions

Table 4.3: Example of users and permissions in a Unix-like system

To implement this in JavaScript, permissions are typically defined as constants:

```
const READ = 4;      // 00000100
const WRITE = 2;     // 00000010
const EXECUTE = 1;   // 00000001
```

These values can be combined using the bitwise OR operator:

```
let permissions = READ | WRITE;  // 00000110
```

And checked using the bitwise AND operator:

```
if (permissions & READ) {
    console.log("Read permission granted");
}
```

Example: Representing a system state

Bitwise operations can also be used to represent the state of a system as a combination of flags. For example:

```
const STATE_IDLE = 1;      // 0001
const STATE_RUNNING = 2;   // 0010
const STATE_ERROR = 4;     // 0100

let systemState = STATE_RUNNING | STATE_ERROR; // 0110
```

This representation allows multiple states to be tracked simultaneously in a compact and efficient way.

&: The Bitwise AND Operator

The `&` operator performs a bitwise AND between two numbers. Each operand is first converted to a 32-bit integer, and the operation is applied bit by bit.

This operator is useful for low-level checks such as:

- parity checks,
- power-of-two checks,
- permission and flag handling.

Parity check A common way to check whether a number is even is:

```
n % 2 === 0
```

The modulo operator involves a division, which is generally more expensive than bitwise operations at the hardware level. Although modern CPUs optimize division, bitwise operations remain simpler and closer to the hardware.

A more efficient approach relies on the binary representation of numbers. Consider:

$$
\begin{aligned}
(1)_{10} &= (00000001)_2 \\
(3)_{10} &= (00000011)_2 \\
(5)_{10} &= (00000101)_2 \\
(7)_{10} &= (00000111)_2 \\
&\vdots
\end{aligned}
$$

$$
\begin{aligned}
(0)_{10} &= (00000000)_2 \\
(2)_{10} &= (00000010)_2 \\
(4)_{10} &= (00000100)_2 \\
(6)_{10} &= (00000110)_2 \\
&\vdots
\end{aligned}
$$

All odd numbers have the least significant bit (LSB) set to `1`, while even numbers have it set to `0`. Therefore, checking the parity of a number reduces to checking its LSB.

This can be done using:

```
n & 1 === 0
```

If the result is:

- `0` → the number is even,
- `1` → the number is odd.

Power-of-two check A naive approach to check whether a number is a power of two consists of repeatedly dividing the number by 2 until an odd value is obtained. If the result is 1, then the original number is a power of two.

A more efficient method uses bitwise operations.

First, observe the binary representation of powers of two:

$$\begin{aligned}(1)_{10} &= (00000001)_2\\(2)_{10} &= (00000010)_2\\(4)_{10} &= (00000100)_2\\(8)_{10} &= (00001000)_2\\&\vdots\end{aligned}$$

A power of two has exactly one bit set to `1`. For example, $(6)_{10} = (00000110)_2$ has two bits set, so it is not a power of two.

The check can be performed as follows:

```
(n > 0) && (n & (n - 1)) === 0
```

Why it works:

- If n is a power of two, it has a single `1` bit.
- Subtracting 1 flips that bit to `0` and sets all lower bits to `1`.
- Therefore, $n \mathbin{\&} (n-1) = 0$.

Example:

$$\begin{array}{lll}(8)_{10} & (00001000)_2 & \& \\(7)_{10} & (00000111)_2 & = \\\hline(0)_{10} & (00000000)_2 & \end{array}$$

For a non-power of two:

$$\begin{array}{lll}(6)_{10} & (00000110)_2 & \& \\(5)_{10} & (00000101)_2 & = \\\hline(4)_{10} & (00000100)_2 & \end{array}$$

The condition $n > 0$ is required because the expression also evaluates to true for $n = 0$.

Checking permissions using bitwise AND Bitwise AND is commonly used to check whether a specific flag is set.

```
let permission = 6; // 00000110 (READ + WRITE)

if (permission & READ) {
    // This code is executed because the user has read permission
}

if (permission & WRITE) {
    // This code is executed because the user has write permission
```

```
}

if (permission & EXECUTE) {
    // This code is NOT executed because the user does not have execute permission
}
```

|: The Bitwise OR Operator

The | operator performs a bitwise OR between two numbers. Each operand is first converted to a 32-bit integer, and the operation is applied bit by bit.

For example:

$$\begin{array}{lll} (6)_{10} & (00000110)_2 & | \\ (5)_{10} & (00000101)_2 & = \\ \hline (7)_{10} & (00000111)_2 & \end{array}$$

Assigning permissions and state components Bitwise OR is commonly used to combine flags. For example:

```
let permission = READ | WRITE; // 00000110
```

Similarly, it can be used to represent a system state as a combination of independent components:

```
let state = COMPONENT_1 | COMPONENT_2 | ... | COMPONENT_N;
```

^: The Bitwise XOR Operator

The ^ operator performs a bitwise XOR (exclusive OR) between two numbers.

For example:

$$\begin{array}{lll} (6)_{10} & (00000110)_2 & \wedge \\ (5)_{10} & (00000101)_2 & = \\ \hline (3)_{10} & (00000011)_2 & \end{array}$$

Flipping individual bits The XOR operator can be used to flip specific bits using a mask.

```
x = x ^ 1;
```

This operation flips only the least significant bit of `x`. More generally, XOR can flip multiple bits depending on the mask used.

Swapping two variables without a temporary variable XOR can be used to swap two variables without a temporary variable:

```
let a = 5; // 00000101
let b = 4; // 00000100

a = a ^ b; // 00000001
b = b ^ a; // 00000101
a = a ^ b; // 00000100
```

Although this technique works, it is generally discouraged in modern JavaScript because it reduces readability and offers no practical performance benefit over using a temporary variable.

Checking if two numbers have opposite signs XOR can be used to check whether two integers have opposite signs:

```
(x ^ y) < 0
```

This works because, in two's complement representation, the most significant bit (MSB) represents the sign:

- MSB = 0 → positive number
- MSB = 1 → negative number

If two numbers have different signs, their XOR will have the MSB set to 1, resulting in a negative number.

Compared to:

```
(x * y) < 0
```

the XOR-based approach avoids multiplication. However, in modern JavaScript engines, such micro-optimizations are usually unnecessary, as the JIT compiler already produces efficient machine code.

Implementing simple encryption algorithms XOR has a useful self-inverse property:

$$x \oplus k \oplus k = x$$

This can be used to implement simple (though not secure) symmetric encryption:

```
// Encryption
let y = x ^ key;

// Decryption
let z = y ^ key; // z = x
```

Detecting changes in data XOR can also be used to detect differences between two values:

```
if ((x ^ y) === 0) {
    console.log("The data is unchanged");
} else {
    console.log("The data changed");
}
```

If `x` and `y` are equal, their XOR is 0; otherwise, it is non-zero.

~: The Bitwise NOT Operator

The ~ operator flips all bits of a number. In JavaScript, operands are first converted to 32-bit signed integers using two's complement representation.

For example:

```
let x = 5;   // 00000101
let y = ~x;  // 11111010  -> -6
```

Important property:

```
~x = -(x + 1)
```

This follows from the two's complement representation of signed integers.

Relation with XOR The bitwise NOT operation can be seen as a special case of XOR with a mask of all 1s:

```
~x = x ^ (-1)
```

since `-1` is represented as a sequence of all 1s in two's complement.

<<: The Bitwise Left Shift Operator

The `<<` operator shifts bits to the left, inserting zeros on the right and discarding bits that overflow on the left. For example:

```
let x = 5;      // 00000101
let y = x << 2; // 00010100  -> 20
```

Multiplication by powers of 2 Left shift can be used to multiply a number by a power of 2:

$$\texttt{x << n = x * } 2^n$$

Important note:

- This holds only if no significant bits are lost (overflow),
- JavaScript uses 32-bit signed integers for bitwise operations.

>>: The Sign-Preserving Right Shift

The `>>` operator shifts bits to the right, preserving the sign by replicating the most significant bit (MSB). For example:

```
let x = -128;   // 10000000 (8-bit example)
let y = x >> 1; // 11000000  -> -64
```

Division by powers of 2 Right shift can approximate division by powers of 2:

$$\texttt{x >> n} \approx \texttt{floor(x / } 2^n\texttt{)}$$

Important:

- For negative numbers, the result is rounded toward negative infinity,
- This is not always identical to standard division.

>>>: The Zero-Fill Right Shift

The `>>>` operator shifts bits to the right and inserts zeros from the left, ignoring the sign. For example:

```
let x = -128;
let y = x >>> 1; // large positive number
```

Important:

- The result is always a non-negative integer,
- This operator treats the number as an unsigned 32-bit value.

Unlike `>>`, this operator does not preserve the sign and is therefore not suitable for arithmetic division of negative numbers.

Bitwise Assignment Operators

JavaScript provides shorthand operators that combine bitwise operations with assignment:

- `&=` (AND assignment)
- `|=` (OR assignment)
- `^=` (XOR assignment)
- `<<=` (left shift assignment)
- `>>=` (right shift assignment)
- `>>>=` (zero-fill right shift assignment)

4.1.4 Clarification of the Ternary Operator

The ternary operator provides a compact way to assign a value based on a condition, without using an explicit `if-else` statement.

Consider the following example:

```
let x;

if (condition) {
    x = 5;
} else {
    x = 10;
}
```

In this case, the `if-else` statement is only used to assign a value to `x`, without any additional logic. This can be rewritten more concisely using the ternary operator:

```
x = condition ? 5 : 10;
```

Syntax intuition

The ternary operator follows this structure:

```
condition ? value_if_true : value_if_false
```

A useful way to read it is:

> "If the condition is true, use the first value; otherwise, use the second value."

Advantages and limitations

The ternary operator improves:

- code conciseness,
- readability (when used for simple expressions).

However:

- it can reduce readability if nested or used with complex expressions,
- it should be avoided when the logic becomes non-trivial.

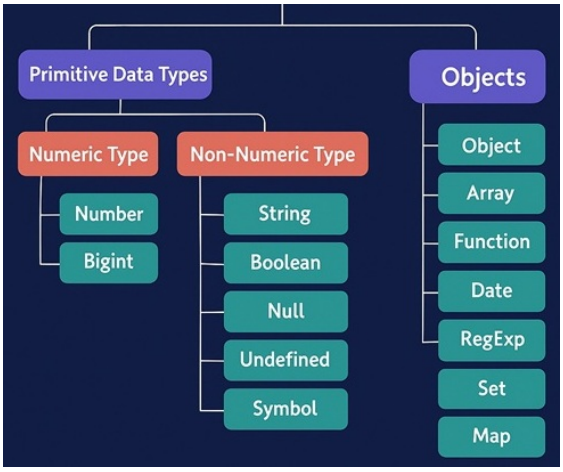

Figure 4.1: JavaScript data types

Performance considerations

Using the ternary operator does **not** provide meaningful performance improvements compared to an equivalent `if-else` statement.

Modern JavaScript engines (via JIT compilation) optimize both forms effectively, often generating very similar machine code.

Therefore:

- the choice between ternary and `if-else` should be based on readability,
- not on performance considerations.

On code size and network transfer

While the ternary operator can reduce the number of characters in the source code, this has negligible impact in practice because:

- JavaScript files are typically minified and compressed,
- network transfer size differences are minimal.

Thus, the main advantage of the ternary operator is expressiveness rather than efficiency.

4.1.5 Clarification of Type Operators

To understand the difference between the `typeof` and `instanceof` operators, it is important to recall that JavaScript distinguishes between **primitive values** and **objects** (see figure 4.1).

JavaScript defines the following primitive types:

- `number`
- `bigint`
- `string`
- `boolean`

- `undefined`
- `symbol`
- `null`

All other values are objects.

Understanding the `typeof` Operator

The `typeof` operator returns a string describing the type of a value.

```
// Primitive values
typeof 10.0;            // 'number'
typeof NaN;             // 'number'
typeof 1234n;           // 'bigint'
typeof "Hello";         // 'string'
typeof true;            // 'boolean'
typeof x;               // 'undefined'
typeof Symbol();        // 'symbol'

// Objects
typeof {key:'value'};   // 'object'
typeof [1,2,3];         // 'object'
typeof new Map();       // 'object'
typeof new Set();       // 'object'

// Function
typeof function(){};    // 'function'

// Special case
typeof null;            // 'object'
```

Why does `typeof null` return `'object'`? This is a well-known historical bug in JavaScript. In early implementations, values were represented with type tags, and the tag for objects was `0`. The value `null` was represented as a null pointer (`0x00`), which was mistakenly interpreted as an object.

This behavior was never corrected for backward compatibility reasons and remains part of the language.

Understanding the `instanceof` Operator

The `instanceof` operator checks whether an object is derived from a constructor's prototype.

```
// Objects
let obj = {key:'value'};
obj instanceof Object;      // true

let arr = ['apple', 'orange'];
arr instanceof Object;      // true
arr instanceof Array;       // true
```

```
let fun = function(){};
fun instanceof Object;      // true
fun instanceof Function;    // true

let date = new Date();
date instanceof Object;     // true
date instanceof Date;       // true

let regex = /e/;
regex instanceof RegExp;    // true

let set = new Set();
set instanceof Object;      // true
set instanceof Set;         // true

let map = new Map();
map instanceof Object;      // true
map instanceof Map;         // true

// Primitive values
5 instanceof Number;        // false
5 instanceof Object;        // false

// Wrapper objects
let num = new Number(5);
num instanceof Number;      // true
num instanceof Object;      // true
```

Key Differences Between `typeof` and `instanceof`

- `typeof` works on both primitives and objects and returns a string.
- `instanceof` returns a boolean value.
- `instanceof` works only on objects and checks the prototype chain.
- `typeof` is useful for primitive type checking.
- `instanceof` is useful for checking object types and class relationships.

Is `Object` a superclass in JavaScript?

JavaScript is not class-based in the same way as Java, but it uses a **prototype-based inheritance model**.

However, in practice:

- most objects inherit from `Object.prototype`,
- therefore, `obj instanceof Object` is usually `true`.

This makes `Object` behave similarly to a superclass, although the underlying mechanism is different.

Wrapper Objects in JavaScript

JavaScript provides wrapper objects such as:

- `Number`
- `String`
- `Boolean`

These allow primitive values to be temporarily treated as objects. For example:

```
let str = "hello";
str.length; // works due to temporary wrapping
```

Using the dot notation to call a method of that primitive data type works because, under the hood, JavaScript creates a temporary and volatile wrapper object containing the same value of the variable. In this example in this specific JavaScript does something like:

```
new String("hello").length;
```

After using the wrapper object, JavaScript destroys it because it is not necessary anymore. This process is called **autoboxing**.

Important:

- Primitive values are not objects.
- Wrapper objects (e.g., `new Number(5)`) should generally be avoided.

Why should wrapper objects be avoided? Although wrapper objects provide a mechanism to treat primitive values as objects, their explicit usage (e.g., `new Number(5)`, `new String("hello")`) is generally discouraged for several reasons.

- **Unexpected behavior in comparisons:** Wrapper objects are fundamentally different from primitive values. This can lead to unintuitive results when using comparison operators.

  ```
  let a = 5;
  let b = new Number(5);

  a == b;   // true
  a === b;  // false
  ```

  The strict equality operator returns `false` because `a` is a primitive value while `b` is an object.

- **Incorrect boolean evaluation:** All objects in JavaScript are truthy (this topic will be discussed later), regardless of their internal value. This can lead to logical errors.

  ```
  if (new Boolean(false)) {
      console.log("This will be executed");
  }
  ```

  Even though the wrapped value is `false`, the condition evaluates to `true` because the object itself is truthy.

- **Unnecessary overhead:** Wrapper objects introduce additional complexity compared to primitive values. They include internal properties and a prototype chain, which makes them heavier than primitives in terms of memory and execution overhead.
- **Temporary vs explicit wrapping:** JavaScript already performs automatic wrapping (autoboxing) when needed. Explicitly creating wrapper objects is therefore redundant and may introduce confusion.
- **Loss of simplicity and readability:** Using primitives keeps the code simpler and more predictable, while wrapper objects introduce behaviors that are not immediately obvious.

In practice, primitive values should always be preferred over their corresponding wrapper objects. Wrapper classes exist mainly for internal use by the language (via autoboxing) and for historical reasons, rather than for direct use in application code.

4.2 Control Flow Statements

Control flow statements allow you to direct the execution path of your code. Their syntax is virtually identical to that of languages like C and Java.

4.2.1 Conditional Statements

Conditional Statements (`if/else`, `switch`) execute a block of code only if a specified condition is met.

if / else Statements: Executes code based on a condition.

```
if (score >= 90) {
    console.log("Grade: A");
} else if (score >= 70) {
    console.log("Grade: B");
} else {
    console.log("Grade: C");
}
```

switch Statement: Chooses between multiple cases.

```
switch (day) {
    case 1: console.log("Monday"); break;
    case 2: console.log("Tuesday"); break;
    default: console.log("Other day");
}
```

4.2.2 Loop Statements

Loop Statements (`while`, `do-while`, `for`) execute a block of code repeatedly as long as a condition remains true or for a specified number of iterations.

while Loop: Repeats code as long as a condition is true.

```
while (x < 10) {
    x++;
}
```

do-while Loop: Repeats code as long as a condition is true but the condition is checked at the end of the first iteration.

```
do {
    x++;
} while (x < 3);
```

for Loop: Repeats code a set number of times.

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}
```

4.2.3 Truthy and Falsy Values

In JavaScript, truthy and falsy values are concepts related to implicit boolean evaluation. Every value can be converted to either `true` or `false` when used in a boolean context (e.g., conditionals or logical expressions).

A **truthy value** is a value that evaluates to `true`, while a **falsy value** evaluates to `false`. Any value that is not explicitly falsy is considered truthy.

Truthy Values

The following are examples of truthy values:

- `true`
- Non-zero numbers: `1`, `-1`, `10.0`
- Non-zero BigInt values: `10n`
- Non-empty strings: `"Hello"`, `"0"`, `" "`
  Note that:
  - `"0"` is truthy because it is a non-empty string,
  - `" "` (a space) is also truthy because it is not empty.
- Objects: `{key: "value"}`, `{}`
- Arrays: `[1,2,3]`, `[]`
- Functions: `function() {}`, arrow functions, etc.
- Dates: `new Date()`
- Symbols: `Symbol()`

Important: All objects are truthy, even if they are empty.

Falsy Values

JavaScript has a small and well-defined set of falsy values:

- `false`
- `0` and `-0`
- `0n` (BigInt zero)
- `""` (empty string)
- `null`
- `undefined`
- `NaN`
- `document.all`

Why is `document.all` falsy? `document.all` is a legacy feature introduced for compatibility with very old browsers. It behaves like an object, but it is treated as falsy for historical reasons. Additionally, `typeof document.all` returns `"undefined"`, making it a special case in the language.

Explicit Boolean Conversion

When in doubt, a value can be explicitly converted to boolean using:

```
Boolean(value)
```

or using double negation:

```
!!value
```

For example:

```
console.log(Boolean("")); // false
console.log(Boolean(10)); // true

console.log(!!""); // false
console.log(!!10); // true
```

Default Values Using Logical OR (||)

The `||` operator is commonly used to assign default values when a variable is falsy:

```
let value = "";
let displayValue = value || "Default value";
console.log(displayValue); // "Default value"
```

However, this approach may lead to unintended behavior when valid falsy values (such as `0` or `""`) are expected.

Nullish Coalescing vs Logical OR

Nullish values (`null` and `undefined`) are a subset of falsy values.

```
let value = null;
let displayValue = value || "Default value";
console.log(displayValue); // "Default value"

let value2 = "";
let displayValue2 = value2 ?? "Default value";
console.log(displayValue2); // ""
```

The `??` operator only checks for nullish values, while `||` checks for all falsy values. This distinction is important when:

- `0` is a valid value (e.g., counters),
- `""` is meaningful (e.g., string building),
- only missing values (`null`/`undefined`) should trigger a default.

Safe Property Access

Truthy/falsy checks can be used to avoid runtime errors:

```
let object = {};

if (object && object.field) {
    console.log("Object and field exist");
} else {
    console.log("At least one of the two does not exist");
}
```

In this example:

- `object` is truthy,
- `object.field` is `undefined` (falsy),

so the condition evaluates to `false`.

Avoiding Explicit Comparisons

Truthy and falsy values allow concise conditions:

```
let variable;

if (!variable) {
    console.log("Value is falsy.");
}
```

Here, `variable` is `undefined`, which is falsy, so the condition evaluates to `true`.

5 Functions: Reusable Blocks of Code

A function is a reusable block of code designed to perform a specific task. In JavaScript, functions are "first-class citizens," which means they are treated like any other object. They can be assigned to variables, passed as arguments to other functions, and returned from other functions.

5.1 Function Definition and Usage

There are two primary ways to define a function: as a declaration or as an expression.

1. **Function Declaration:** This is the traditional way to define a named function.
2. **Function Expression:** This involves creating a function (often anonymous) and assigning it to a variable.

The key difference between these two forms lies in their **hoisting** behavior. Function declarations are fully hoisted, meaning they are loaded into memory before any code is executed and can be called before they appear in the code. In contrast, with function expressions, only the variable declaration (`var greet`) is hoisted (and initialized with `undefined`), not the function assignment itself. Calling the function before the assignment line will result in an error.

Because functions are objects in JavaScript, they can be passed as arguments to other functions. A function that is passed as an argument to another function is known as a **callback function**. The outer function can then invoke this callback to complete a routine or action at the appropriate time.

For example:

```
// Function expression to be used as a callback
var printMessage = function(max) {
    var message = "";
    for (var i = 0; i < max; i++) {
        message += "hello ";
    }
    return message;
}

// Function declaration accepting a callback
function hello(callbackFunc) {
    return "Welcome " + callbackFunc(3);
}

// Pass printMessage as a callback to the hello function
hello(printMessage);
```

```
OUTPUT
Welcome hello hello hello
```

This powerful feature of treating functions as data is central to many advanced patterns in JavaScript, especially when working with complex data structures like arrays and objects.

6 Working with Data Structures: Arrays and Objects

Arrays and objects are the primary data structures for organizing collections of data in JavaScript. Arrays store ordered lists of values, while objects store collections of key-value pairs. Mastering their properties and methods is essential for any form of data manipulation.

6.1 Arrays

A JavaScript array is a structured, ordered sequence of values.

```
let names = ["Antonio", "Giuseppe", "Cosimo"];
```

The key characteristics of an array make it a flexible and powerful tool:

- **Dynamic Sizing:** Arrays can grow or shrink at runtime.

  ```
  let names = new Array();
  names[0] = "Antonio";
  names[1] = "Giuseppe";
  names[2] = "Cosimo";
  ```

- **Sparse Nature:** Arrays can contain empty slots. Those elements occupy no memory, making arrays efficient for storing data with gaps.
- **Heterogeneous Elements:** An array can contain elements of mixed data types (e.g., numbers, strings, and even other arrays).

  ```
  let colors = ["Yellow", "Red", , , "Blue"];
  ```

  In this example, the array has 5 elements but the elements with indices 2 and 3 are **empty slots** (not explicitly assigned). Accessing them returns `undefined`.

  We can also create sparse arrays manually:

  ```
  let array_test = new Array();
  array_test[0] = "var";
  array_test[1] = "bar";
  array_test[3] = "filler";
  array_test[4] = "foo";

  console.log(array_test);
  // ['var', 'bar', , 'filler', 'foo']
  ```

- **Nesting possibility:** JavaScript does not have true multidimensional arrays, but it supports **arrays of arrays**. For example:

```
let matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
];

matrix[0][1]; // 2
```

  Important clarification: Using multiple indices does not automatically create nested arrays.

  Consider the following experiment:

```
let test = ['Top element 0', 'Top element 1', 'Top element 2'];

test[0][0] = 'Nested element 0-0';
test[0][1] = "Nested element 0-1";
test[2][2] = "Nested element 2-2";
```

  At first glance, it may seem that we are working with a two-dimensional array. However, this is not the case.

  - Each element of `test` is a **string**, not an array.
  - Expressions like `test[0][0]` are interpreted as:

```
("Top element 0")[0]
```

  - This means we are accessing a **character** of a string, not a second dimension.

  Important: Strings in JavaScript are **immutable**. Therefore, assignments like:

```
test[0][0] = 'Nested element 0-0';
```

  do not actually modify the string. The operation is silently ignored, even though the console may display the assigned value.

- **Easy length access:** The `.length` property returns the number of elements in the array. Consider the following experiment:

```
let test = ['Top element 0', 'Top element 1', 'Top element 2'];

test.length;     // 3
test[0].length;  // 13
test[2].length;  // 13
```

  - `test.length = 3` because the array contains 3 elements.
  - `test[0].length = 13` because `test[0]` is a string:

```
"Top element 0"
```

which contains 13 characters.

Key takeaway:

- The `length` of an array counts its elements.
- The `length` of a string counts its characters.
- Accessing `array[i][j]` only makes sense if `array[i]` is itself an array.

Arrays come with a rich set of built-in methods for manipulation, including:

- `.sort()`: Sorts the elements of an array in place. By default, it sorts alphabetically.
- `.sort(compare_function)`: Orders the array according to a comparison function between elements defined by the user. For example:

```
myArray.sort(function(a, b) { return a - b; })
```

- `.reverse()`: Reverses the order of the elements of an array in place.
- `.push()`: Adds one or more elements to the end of an array and increases its size.
- `.pop()`: Removes the last element from an array and returns that element. This operation reduces the size of the array.
- `.toString()`: Returns a string containing the values of the array separated by commas.

For iteration, the `forEach()` and `map()` methods are particularly useful:

- **`forEach()`**: Executes a provided callback function once for each array element. It does not create a new array and can modify its elements. This method is used in situations in which we don't have to create another array even if we don't have to modify it.
- **`map()`**: Creates a **new array** populated with the results of calling a provided callback function on every element in the calling array. This is ideal for transforming data from one form to another.

6.1.1 Example of `forEach()` usage

In this example we show the elements contained in an array one by one.

```
let months = ["Jan", "Feb", "March", "April"];
months.forEach(function(element, index, array){
    console.log(element.toUpperCase());
});

OUTPUT:
    JAN
    FEB
    MARCH
    APRIL
```

Note:

- The internal callback function can have at most 3 parameters:
  1. `element`: The value of the current array element;
  2. `index`: The index of the current array element;
  3. `array`: A reference to the whole array. This value is useful if we want to modify the original array elements.
- JavaScript has a placeholder character so calling functions ignoring a parameter is possible. It is possible to call the `forEach()` function as follows:

```
let months = ["Jan", "Feb", "March", "April"];
months.forEach(function(_, index, array){
    console.log(array[index].toUpperCase());
});
```

6.1.2 Example of `map()` usage

This example is similar to the previous one: given an array of months we create another array containing the name of the months in upper case.

```
let months = ["Jan", "Feb", "March", "April"];
let newMonths = months.map( function(element, index, array){
    return element.toUpperCase();
});

OUTPUT:
    newMonths = ["JAN", "FEB", "MARCH", "APRIL"]
```

Notice that the internal callback function has the same parameters as the internal callback function of the `forEach()` method but, unlike the previous method, the `map()` method returns a new array. In fact, the internal callback function of this method has a return statement and the returned value is store in a variable.

To better understand what `map()` does, consider its conceptual equivalent written as a `for` loop:

```
// This for loop demonstrates what the map() method does internally.
let originalArray = [1, 2, 3];
let newArray = [];

for (let i = 0; i < originalArray.length; i++) {
    // A function is applied to each element, and the result is pushed to th
    newArray.push(originalArray[i] * 2);
}
// newArray is now [2, 4, 6]
```

6.2 Objects

A JavaScript object is an unordered collection of key-value pairs, where keys are strings or Symbols, and values can be any data type.

Objects are typically created using object literal syntax. Properties can be accessed using either dot notation or bracket notation.

```
// Creating an object literal
let person = {
  name: "Alice",
  age: 30,
  isStudent: false,
  greet: function() {
    console.log("Hello, " + this.name);
  }
};

// Accessing properties
console.log(person.name);       // Outputs: Alice (dot notation)
console.log(person["age"]);   // Outputs: 30 (bracket notation)

// Calling an object's method
person.greet();                 // Outputs: Hello, Alice

// Deleting a property
delete person.isStudent;
console.log(person.isStudent);  // Outputs: undefined
```

6.2.1 Creating an object

In JavaScript, objects can be created in 3 ways:

1. **Using an object constructor:** An object constructor is a special function with the purpose of creating an initializing an object. For example:

```
function Person(n, c){
    // The "this" keyword is mandatory
    this.name = n;
    this.surname = c;
}
...
let bruce = new Person("Bruce", "Springsteen");
```

2. **Using an object initializer instruction:** In JavaScript, objects are unordered collections of key-value pairs enclosed in curly braces and can be created this way. For example:

```
let obj = {property_1: value_1, property_2: value_2,
..., property_n, value_n};
let victor = {name: "Victor", surname: "Hugo"};
```

3. **Creating an empty object and assigning properties to it:** In JavaScript, objects are dynamic. We can create them as empty associative arrays and add properties later or we could also remove existing properties as in the fist example of this section. Here is an example of empty object initialization and subsequent assignment:

```
let p = new Object();
p.name = "Floriano";
```

6.2.2 Methods

When a function is stored as a property of an an object, it is called a **method**. To create a method, we can either create a function internal to the collection in the creation phase as we have seen in the first example of this section or we can add a previously created function dynamically after the creation of the object. Here is an example of the second way:

```
function setYearOfBirthday(a){
    this.year = a;
}
function Person(n){
    this.name = n;
    this.setYear = setYearOfBirthday;
}
...
let boss = new Person("Bruce Springsteen");
boss.setYear(1949);
/*
    Now the boss object has the following properties:
        year = 1949
        name = Bruce Springsteen
*/
```

It's important to notice that the assignment that turns the previously created function into a method of the object only uses the identifier of the function **without the parenthesis!**. This is because we want to assign to the property `setYear` an handler to the function `setYearOfBirthday` and not a value returned by the latter. If we used the parenthesis we would have called the function to make it return a value and JavaScript would have raised an error when trying to run the command `boss.setYear(1949);` saying that `boss.setYear` is not a function. The value returned by the function `setYearOfBirthday()` without arguments is `undefined`.

6.2.3 Common objects

JavaScript has a set of commonly used objects which are listed in table 6.1.

JSON stringification is the process of turning a JSON object into a string:

```
let json_example = {key:'value', var:undefined, var2:null};
json_string = JSON.stringify(json_example);
\\ Prints: '{"key":"value","var2":null}'
```

Notice that the undefined value is removed during the stringification process.

When working with object methods, you will frequently encounter the `this` keyword, which refers to the object a method is being called on. Its behavior, however, can be complex and is a common source of confusion.

Object	Description	Examples
Object	The base object from which all other objects inherit.	`let obj = new Object();`
Array	A collection of ordered elements indexed by numbers.	`let arr = [1, 2, 3];`
Function	A block of code designed to perform a particular task.	`function myFunc() { }`
Date	Used to work with dates and times.	`let now = new Date();`
RegExp	Represents regular expressions for pattern matching in strings.	`let regex = /abc/;`
Math	Provides mathematical constants and functions.	`let pi = Math.PI;`
JSON	Provides methods for parsing and stringifying JSON data.	`let jsonString = JSON.stringify(obj);`

Table 6.1: Commonly used objects in JavaScript

7 Advanced Concepts: this and Symbol

As you progress with JavaScript, you will encounter concepts that, while more complex, are critical for writing robust and modern code. The `this` keyword and the `Symbol` data type are two such concepts that solve common but nuanced problems in the language.

7.1 The `this` Keyword: Understanding Context

The `this` keyword is one of the most important and, at the same time, most confusing concepts in JavaScript.

Definition: The value of `this` depends on **how a function is called**, not on where it is defined.

7.1.1 Strict mode vs. non-strict mode

JavaScript has two modes of operation: strict mode and non-strict mode. These modes affect how JavaScript code is executed, and understanding the differences between them is crucial for writing secure and reliable code.

- **Strict mode** is a subset of JavaScript that provides better error checking and enforces stricter rules for coding. It was introduced in ECMAScript 5 (ES5), which is the fifth edition of the ECMAScript language specification. When strict mode is enabled, the JavaScript engine checks for syntax errors and runtime errors that would otherwise go unnoticed in non-strict mode. This makes it easier for developers to catch errors early in the development process, resulting in fewer bugs and better code quality.
- **Non-strict mode**, on the other hand, is the default mode of operation in JavaScript. In this mode, the JavaScript engine allows for more flexible coding and provides fewer restrictions. This means that some code that would cause errors in strict mode may execute without any issues in non-strict mode. While this may seem advantageous at first, it can lead to unexpected behavior and security vulnerabilities.

To enable strict mode in JavaScript, you simply add the string `"use strict";` at the beginning of a script or function.

To the ends of our exposition, we need to consider that in non-strict mode, the `this` keyword inside a function that is not invoked as a method of an object refers to the global object. In strict mode, however, the `this` keyword is undefined in such cases. This helps prevent accidental modification of the global object and makes it easier to write secure code.

7.1.2 The "left of the dot" rule

A useful heuristic to understand `this` is:

> *Look at what is to the left of the dot when a function is called.*

```
let obj = {
    name: "Antonio",
    greet: function() {
        console.log(this.name);
    }
};

obj.greet(); // "Antonio"
```

Here:

- the function is called as `obj.greet()`
- `this` refers to `obj`

7.1.3 Default binding (global context)

If a function is called without an object, `this` refers to the global object (or `undefined` in strict mode):

```
function show() {
    console.log(this);
}

show(); // window (browser) or undefined (strict mode)
```

7.1.4 The `this` problem inside callbacks

Consider the following example (from the lecture slides):

```
var period = {
    message: "Hello",
    months: ["Jan", "Feb", "March", "April"],
    show: function() {
        this.months.forEach(function(month) {
            console.log(this.message + " " + month);
        });
    }
};

period.show();
```

Output:

```
undefined Jan
undefined Feb
undefined March
undefined April
```

Explanation:

- `this` inside `show()` refers to `period`
- but `this` inside the `forEach` callback refers to the **global object**

This happens because:

- the callback function is called **without an explicit object**
- therefore, it loses the original `this` binding

7.1.5 Solutions to preserve `this`

Solution 1: Store `this` in a variable

```
var period = {
    message: "Hello",
    months: ["Jan", "Feb", "March", "April"],
    show: function() {
        let _this = this;

        this.months.forEach(function(month) {
            console.log(_this.message + " " + month);
        });
    }
};
```

Explanation:

- `_this` stores the correct reference
- closures allow access to it inside the callback

Solution 2: Use `bind()`

```
var period = {
    message: "Hello",
    months: ["Jan", "Feb", "March", "April"],
    show: function() {
        this.months.forEach(function(month) {
            console.log(this.message + " " + month);
        }.bind(this));
    }
};
```

Explanation:

- `bind(this)` creates a new function
- the new function has `this` permanently bound to `period`

Solution 3: Use arrow functions (recommended)

```
var period = {
    message: "Hello",
    months: ["Jan", "Feb", "March", "April"],
    show: function() {
        this.months.forEach((month) => {
```

```
            console.log(this.message + " " + month);
        });
    }
};
```

Output:

```
Hello Jan
Hello Feb
Hello March
Hello April
```

Explanation:

- arrow functions do **not** have their own `this`
- they inherit `this` from the surrounding scope (lexical scoping)

7.1.6 Key observations

- `this` is determined at **call time**, not at definition time
- normal functions have their own `this`
- arrow functions inherit `this` from the outer scope
- callbacks (like in `forEach`) often lose the original `this`

7.1.7 Important takeaway

In JavaScript, understanding `this` means understanding how a function is invoked.

7.2 The Symbol Type: Ensuring Uniqueness

The `Symbol` primitive type was introduced to solve a specific problem: **preventing property key collisions in objects**. When multiple libraries or different parts of a large application add properties to a shared object using string keys, there is a risk that they will accidentally use the same key name and overwrite each other's data.

Symbols are guaranteed to be unique, which eliminates this risk.

- A new Symbol is created by calling `Symbol()`. Each call produces a new, unique value, even if the descriptive string is the same: `Symbol('id') !== Symbol('id')`.
- Symbol-keyed properties are "quasi-private." They are not enumerated in `for...in` loops and are ignored by `JSON.stringify()` and `Object.keys()`, making them ideal for storing internal metadata that should not interfere with normal object operations.

It is important to distinguish between `Symbol()` and `Symbol.for()`:

- **`Symbol()`** always creates a new, unique symbol.
- **`Symbol.for("key")`** searches a global symbol registry for a symbol with the given key. If one exists, it returns it; otherwise, it creates a new symbol, adds it to the registry, and then returns it. This allows symbols to be shared across different files or realms.

Mastering advanced concepts like `this` and `Symbol` empowers developers to write more predictable, maintainable, and collision-free code, which is essential for building complex, scalable applications.